

Migrating off Firebase to a fully self-hosted local server (192.168.1.121)

Goal: zero paid/cloud dependency (no Firebase, no Vercel). Backend, database and file storage all run on your own LAN box, 192.168.1.121 . No monthly subscription, no internet dependency for the app to work.

This is a real rewrite, not a config change — the backend currently talks to Firestore (data), Firebase Auth (login/passwords), and Firebase Storage (employee documents) in ~20 files. Every step below is required, not optional polish.

0. What's actually being replaced

Grep of `main/backend` for `db.collection(`, `bucket.`, `runTransaction`, `.batch(`, `auth.*` turned up the real surface area:

Firestore piece	Replacement	Why this choice
Firestore (<code>db.collection(...)</code>)	MongoDB , installed locally	Firestore is document-shaped (arbitrary fields per doc, no fixed schema) — Mongo is the same shape. Moving to Postgres would mean designing real tables/relations for ~18 collections; Mongo lets every existing <code>db.collection('x').doc(id).get()</code> call become <code>db.collection('x').findOne({_id: id})</code> with the surrounding logic untouched. Less rewrite, less risk.
Firestore Auth (<code>auth.createUser/getUserByEmail/updateUser/deleteUser</code> , password verify)	bcrypt + your own users collection	Firestore owns password hashes today in a format you can't export into bcrypt. This is the one piece that isn't a mechanical swap — see step 4.
Firestore Storage (<code>bucket.file(...).save()</code> , signed URLs)	Local disk folder , served statically	<code>multer</code> is already a dependency and already used for the upload itself; no need to add MinIO/S3 for one feature (employee document uploads in <code>hrDeskController.js</code>).
<code>db.runTransaction(...)</code> (sessions.js, approvalController.js, complaintControllerFactory.js)	MongoDB client sessions + transactions	Mongo transactions require the server to run as a (single-node is fine) replica set — see step 1.
<code>db.batch()</code> (complaintControllerFactory.js, securityController.js)	bulkWrite()	Direct equivalent.

Collections found (these all need to exist in Mongo): `users`, `sessions`, `failed_logins`, `audit_logs`, `settings` (singleton docs: `notification_rules`, `action_permissions`, `role_permissions`, `system_config`, `sla_policies`), `hr_complaints`, `it_complaints`, `approvals`, `leave_requests`, `assets`, `departments`, `employees`, `attendance`, `extra hours`, `sent emails`, `tasks`, `projects`, `renders`.

1. Install MongoDB on 192.168.1.121

1. Download **MongoDB Community Server** (MSI installer) from [mongodb.com](https://www.mongodb.com) — free, no account needed.
2. Install it as a Windows Service (installer does this by default) so it auto-starts on reboot.
3. **Enable it as a single-node replica set** — required for transactions (`runTransaction` usage above), otherwise those calls throw at runtime:

- Edit `C:\Program Files\MongoDB\Server\<version>\bin\mongod.cfg` , add:

```
replication:
  replSetName: "rs0"
```

- Restart the MongoDB service (`services.msc` → MongoDB → Restart).
- Run once from `mongosh` :

```
rs.initiate()
```

4. Confirm it's reachable: `mongosh mongodb://localhost:27017` should connect.
5. It listens on `127.0.0.1:27017` by default — fine, since the backend runs on the same box and talks to it over localhost, not the LAN.

2. Swap the Firebase config module for a Mongo one

Replace `main/backend/config/firebase.js` with `main/backend/config/db.js` :

```
const { MongoClient } = require('mongodb');
require('dotenv').config();

const client = new MongoClient(process.env.MONGO_URI || 'mongodb://localhost:27017/fute_portal');
let db;

async function connect() {
  if (db) return db;
  await client.connect();
  db = client.db();
  return db;
}

module.exports = { client, connect, getDb: () => db };
```

Add `mongodb` to `package.json` dependencies (`npm install mongodb`), and remove `firebase-admin` .

`server.js` must `await connect()` before `app.listen(...)` — Firestore connected lazily on first call, Mongo's driver needs an explicit connect.

3. Rewrite every `db.collection(...)` call site

This is the bulk of the work — 19 controllers + `utils/sessions.js` + `utils/auditLog.js` + `utils/notificationRules.js` + `middleware/authMiddleware.js` + `middleware/permissionMiddleware.js` . The translation is mechanical but must be done file by file:

Firestore	MongoDB
<code>collection.doc(id).get()</code> → <code>.exists</code> / <code>.data()</code>	<code>collection.findOne({_id: id})</code> → <code>null</code> if not found, fields are on the object directly
<code>collection.doc(id).set(data)</code>	<code>collection.replaceOne({_id: id}, data, {upsert: true})</code>
<code>collection.doc(id).set(data, {merge: true})</code>	<code>collection.updateOne({_id: id}, {\$set: data})</code>
<code>collection.doc(id).update(data)</code>	<code>collection.updateOne({_id: id}, {\$set: data})</code>
<code>collection.doc(id).delete()</code>	<code>collection.deleteOne({_id: id})</code>
<code>collection.add(data)</code>	<code>collection.insertOne({...data})</code> — capture <code>insertedId</code> (Mongo won't auto-string it like a Firestore doc id; either let it be an <code>ObjectId</code> or set your own <code>_id</code>)
<code>collection.where('field', '==', val).get()</code>	<code>collection.find({field: val}).toArray()</code>
<code>collection.where('field', '>=', val).orderBy('field').limit(n).get()</code>	<code>collection.find({field: {\$gte: val}}).sort({field: 1}).limit(n).toArray()</code>
<code>collection.count().get()</code>	<code>collection.countDocuments(filter)</code>
Firestore doc id (string, e.g. <code>"AST-1006"</code>)	Use the same value as Mongo's <code>_id</code> — keeps <code>assetController.js</code> 's business-id-as-doc-id convention working unchanged

Do this **one controller at a time**, running that controller's routes against a Postman/curl smoke test before moving to the next. Order by risk: `authController.js` and `middleware/authMiddleware.js` first (nothing else works without login), then `complaintControllerFactory.js` (used by `hrRoutes` / `itRoutes` , biggest file), then the rest.

Transactions and batches specifically

- `utils/sessions.js` (`consumeRefreshToken`) and `approvalController.js` (`updateStatus`) and `complaintControllerFactory.js` (`updateStatus`) use `db.runTransaction` . Mongo equivalent:

```
const session = client.startSession();
try {
  await session.withTransaction(async () => {
    // reads/writes here, passing { session } as the last arg to each call
  });
} finally {
  await session.endSession();
}
```

- `complaintControllerFactory.js` (`deleteComplaint`) and `securityController.js` use `db.batch()` . Mongo equivalent: build an array of `{deleteOne: {...}}` / `{updateOne: {...}}` operations and call `collection.bulkWrite(ops)` .

4. Replace Firebase Auth with local password auth

This is the security-sensitive part — don't shortcut it.

Password hashes already stored in Firebase Auth cannot be exported into bcrypt. Firebase uses a scrypt variant with per-project parameters; there is no supported way to re-verify those hashes outside Firebase. Pick one:

- **(Recommended for this size of project)** Force a password reset for every existing user at cutover — email each user a one-time reset link, or have an admin set a temporary password via `superAdminController.js` 's existing "force reset" endpoint once it's rewritten (step below). Simple, no ambiguity, acceptable for an internal company portal.
- Advanced alternative: implement Google's documented scrypt verification algorithm against an exported Firebase Auth hash dump, so old passwords keep working. More work, only worth it if you can't ask users to reset.

Once decided (assume password reset), rewrite:

- `config/firebase.js` 's `auth export` → delete it. Password hashing moves into the `users` collection itself.
- `authController.js` :
 - `register()` : replace `auth.createUser(...)` with `bcrypt.hash(password, 12)` and store the hash directly on the user doc ({ email, passwordHash, full_name, role, ... }), with your own generated `_id` (e.g. `crypto.randomUUID()`) instead of a Firebase uid.
 - `login()` : replace the Identity Toolkit `fetch(...)` call with `bcrypt.compare(password, user.passwordHash)` .
- Delete `IDENTITY_TOOLKIT_BASE` / `IDENTITY_TOOLKIT_KEY` and the `usingEmulator` import entirely.
- `superAdminController.js` : replace `auth.createUser` , `auth.updateUser(uid, {disabled})` , `auth.deleteUser(uid)` , `auth.updateUser(uid, {password})` with, respectively: insert with a bcrypt hash, `updateOne({_id}, {$set: {active: !active}})` (an `active` field already exists in the schema per `authController.js` 's login check), `deleteOne({_id})` , `updateOne({_id}, {$set: {passwordHash: await bcrypt.hash(newPassword, 12)}})` .
- Add `bcrypt` to `package.json` (`npm install bcrypt`).
- Remove `FIREBASE_API_KEY` from `.env` — no longer needed once `verifyPassword()` also switches to `bcrypt.compare` .

JWT-based sessions (`utils/jwt.js` , cookies, `sid` revocation) are unaffected — none of that is Firebase-specific, it stays exactly as-is.

5. Replace Firebase Storage with local disk

`hrDeskController.js` 's `uploadEmployeeDocument` is the only file that touches `bucket` . Replace:

```
// instead of bucket.file(storagePath).save(req.file.buffer, ...) + getSignedUrl
const fs = require('fs/promises');
const path = require('path');

const UPLOAD_ROOT = path.join(__dirname, '..', 'uploads', 'employee-documents');
await fs.mkdir(path.join(UPLOAD_ROOT, id), { recursive: true });
const diskPath = path.join(UPLOAD_ROOT, id, `${docType}-${Date.now()}-${safeName}`);
await fs.writeFile(diskPath, req.file.buffer);
const url = `/uploads/employee-documents/${id}/${path.basename(diskPath)}`;
```

Then in `server.js` , serve that folder statically:

```
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
```

Add `main/backend/uploads/` to `.gitignore` (actual files shouldn't be committed). Employee documents contain PII — this folder is only reachable on your LAN, same as everything else, but make sure the server itself isn't port-forwarded to the internet.

6. Environment variables

New `main/backend/.env` :

```
MONGO_URI=mongodb://localhost:27017/fute_portal
JWT_SECRET=<keep existing value>
SMTP_HOST=... SMTP_PORT=... SMTP_USER=... SMTP_PASS=... # unchanged
HR_EMAIL=... IT_EMAIL=... # unchanged
PORT=5000
```

Remove every `FIREBASE_*` variable — nothing reads them anymore once step 4 is done.

7. Run the backend on the server as a persistent service

Copy `main/backend` to the server, then:

```
cd main/backend
npm install
npm install mongodb bcrypt
npm uninstall firebase-admin
```

Use **PM2** so it survives reboots/crashes (it's the standard, free choice — no need for a hand-rolled Windows service):

```
npm install -g pm2
pm2 start server.js --name fute-backend
pm2 save
pm2-installer # or `pm2 startup` equivalent so it restarts on Windows boot
```

Backend now listens on `192.168.1.121:5000` .

8. Build and serve the frontend

```
cd main/frontend
npm install
npm run build
```

Serve the `dist/` folder from the same box — simplest is `serve` :

```
npm install -g serve
serve -s dist -l 80
```

(Or point IIS at the `dist` folder if you'd rather use IIS, since it's already on Windows.)

Set `main/frontend/.env` 's `VITE_API_BASE_URL=http://192.168.1.121:5000` before building, and rebuild whenever that changes (Vite bakes env vars in at build time).

9. Backups — Firestore gave you this for free, Mongo on one box doesn't

Non-optional for a system holding real employee data. Schedule a daily dump:

1. Create `C:\backups\` folder.
2. Windows Task Scheduler → daily task running:

```
mongodump --uri="mongodb://localhost:27017/fute_portal" --out="C:\backups\%date:~-4,4%%date:~-10,2%%d"
```

3. **Copy those dumps off the box periodically** (external drive, another machine on the LAN) — a backup that lives on the same disk as the live data doesn't survive that disk failing.
4. To restore: `mongorestore --uri="mongodb://localhost:27017/fute_portal" C:\backups\<date>\fute_portal`

10. Network access

- The server is only reachable from `192.168.1.0/24` (your office LAN) unless you explicitly port-forward it — don't, unless you also add TLS and tighten `cors` 's `allowedOrigins` list in `server.js` beyond `localhost`.
- Windows Firewall must allow inbound on port 5000 (backend) and 80 (frontend) for other LAN machines to reach it — allow only from `192.168.1.0/24`, not "Any".

11. Cutover checklist

- ☐ MongoDB installed, running as replica set, reachable via `mongosh`
- ☐ `config/db.js` replaces `config/firebase.js`
- ☐ All 19 controllers + 4 utils/middleware files converted (test each)
- ☐ `authController.js` + `superAdminUserController.js` on bcrypt, no Firebase Auth calls left (`grep -r "require('firebase-admin')"` returns nothing)
- ☐ Existing users notified / reset before old logins are cut off
- ☐ `hrDeskController.js` uploads to local disk, `/uploads` served statically
- ☐ `.env` has no `FIREBASE_*` keys
- ☐ Backend running under PM2, restarts on reboot
- ☐ Frontend built with `VITE_API_BASE_URL` pointed at the server, served
- ☐ Daily `mongodump` scheduled, verified restorable, copied off-box
- ☐ Firewall restricts backend/frontend ports to the LAN subnet